11 – Submodular Maximization

IEOR 145/245

Rajan Udwani

Reminder: Teams for Simulation and Project

- Details and sample projects on bCourses (see “README Project Info” under Files-> Project)

- Use the sheet for forming teams



Practice Problem Set, Problem Set 2

- Practice Problem Set (released) and Problem Set 2 will be released today
  - Due October 15
  - Last two problems connected to next week's lectures

Cardinality Constrained Probability Maximization

Find assortment such that

$$\max_{S \subseteq [n] \mid |\boldsymbol{S}| \leq \boldsymbol{k}} \phi(S)$$

- Optimal assortment?
  - Suppose $n = 10$, can evaluate 2^{10} candidates easily
  - **Larger $\boldsymbol{n}$**? (in reality, n between hundreds to millions)

The Greedy Algorithm

$$\max_{S \subseteq [n] \mid |\boldsymbol{S}| \leq \boldsymbol{k}} \sum_{i \in S} \phi(i, S)$$

- Idea: Pick best but *accounting for past steps*
- Define marginal value function,

$$\Delta\phi(e|S) = \phi(S + \{e\}) - \phi(S)$$

1. **Build solution in $\boldsymbol{k}$ steps**
2. **Always add element with maximum marginal value to current set**

$$\text{Add } \operatorname{argmax}_{e \in [n] \setminus S} \Delta\phi(e|S)$$

Example

- E.g.:$\lambda_{1,\,2,\,3,\,\,4,..10,0} = 0.1,\ \lambda_{11,\,0} = \lambda_{12,\,0} = \lambda_{13,\,0} = \cdots = \lambda_{19,\,0} = 0.09$

- For $k = 10,$ what does Greedy pick?
- **Step 1:** product 1
- **Step 2:** product 11
- **Step 3:** product 12 and so on…

- Greedy picks the optimal solution $\{1,11,12, \ldots 19\}$

Is Greedy Always Good?

$$\Delta\phi(e|S) = \phi(S + \{e\}) - \phi(S)$$

1. Build solution in k steps
2. Always add element with maximum marginal value to current set

- Does it always get the optimal solution?
  - No – we will see an example of suboptimal situation
  - How do we know if Greedy is a “good” algorithm?

Example for Sub-Optimality of Greedy

- E.g.: $\lambda_{1,2,0} = \lambda_{1,3,0} = 0.26,\ \lambda_{2,0} = \lambda_{3,0} = 0.24$

- For $k = 2,$ what does Greedy pick?
- **Step 1:** product 1
- **Step 2:** product 2 (or product 3)

- Overall, Greedy value $\phi(\{1,2\}) = 0.76$
- Optimal solution $\phi(\{2,3\}) = 1$

In a realistic context

- Coffee cart with room for $k=2$ drinks: 1 = latte, 2 = cold brew, 3 = tea
  - 26%: latte, else cold brew; 26%: latte, else tea; 24%: only cold brew; 24%: only tea
- Step 1: latte alone 0.52 > cold brew alone 0.50 = tea alone 0.50
- Step 2: cold brew adds only its loyal 0.24 (the other 26% already buy latte)
  - Greedy: $0.52+0.24=0.76$
- Cold brew + tea serves everyone: $0.26+0.24+0.26+0.24=1$

Problem Structure: Submodularity (and Monotonicity)

The answer to “Is Greedy good and why?”

Set Functions and Monotonicity

- Given a ground set/universe N
- A set function $f: 2^N \rightarrow \mathbb{R}$ maps sets to real values
  - Example 1: Probability function $\phi(S)$ is a set function!
  - Example 2: two news apps $N = \{1,2\}$ then $f: \emptyset, \{1\}, \{2\}, \{1,2\} \rightarrow \mathbb{R}$

$$f(\emptyset) = 0, \qquad f(\{1\}) = 5, \qquad f(\{2\}) = 3, \qquad f(\{1,2\}) = 6$$

app 1 covers 5 topics, app 2 covers 3, together 6 (they share 2 topics)

- **Monotonicity**: $f(B) \leq f(A) \;\; \forall B \subseteq A \subseteq N$ and $f(\emptyset) = 0$

More items lead to more sales, more apps cover more topics

A General Problem: Subset Selection

Subset Selection: $\max_{S \subseteq N,\, S \in \mathcal{F}} f(S)$

- For e.g., $\mathcal{F} =$ cardinality constraint
- Some subset selection problems:
  - Assortment optimization: which products to show $\left(\max_{S \subseteq N,\, S \in \mathcal{F}} \phi(S)\right)$
  - feature selection (ML): which inputs best predict
  - data summarization: which photos best represent an album
  - influence maximization: which subset of people to convince
  - facility location …

A General Problem: Subset Selection

Subset Selection: $\max_{S \subseteq N,\, S \in \mathcal{F}} f(S)$

- For e.g., $\mathcal{F} =$ cardinality constraint
- Some subset selection problems:
  - Assortment optimization
  - feature selection (ML)
  - data summarization
  - influence maximization
  - facility location …

- In general, hard problem to solve
- "Easier" when f has structure,
  1. Modularity
  2. Submodularity

Present in $\phi(S)$

1. Modular Functions

- $f: 2^N \rightarrow \mathbb{R}$ is a modular function if,

$$f(S) = \sum_{e \in S} f(\{e\}) \qquad \forall\, S \subseteq N$$

Value of a set = sum of the values of its items

- Example: $N = \{1,2\}$ then $f: \emptyset, \{1\}, \{2\}, \{1,2\} \rightarrow \mathbb{R}$

$$f(\emptyset) = 0, \qquad f(\{1\}) = 5, \qquad f(\{2\}) = 3, \qquad \boldsymbol{f(\{1,2\}) = 8}$$

The news apps from before are **not** modular

2. Submodular Functions

- Functions that exhibit the following property:

$$f(A \cup \{i\}) - f(A) \leq f(B \cup \{i\}) - f(B) \quad \forall B \subseteq A, i \in N \backslash \mathrm{A}$$

Gain from adding i to the **bigger** set A ≤ gain from adding i to the **smaller** set B

Submodularity implies: $f(A \cup \{i\}) - f(A) \leq f(\{i\})$ (when $B = \emptyset$)

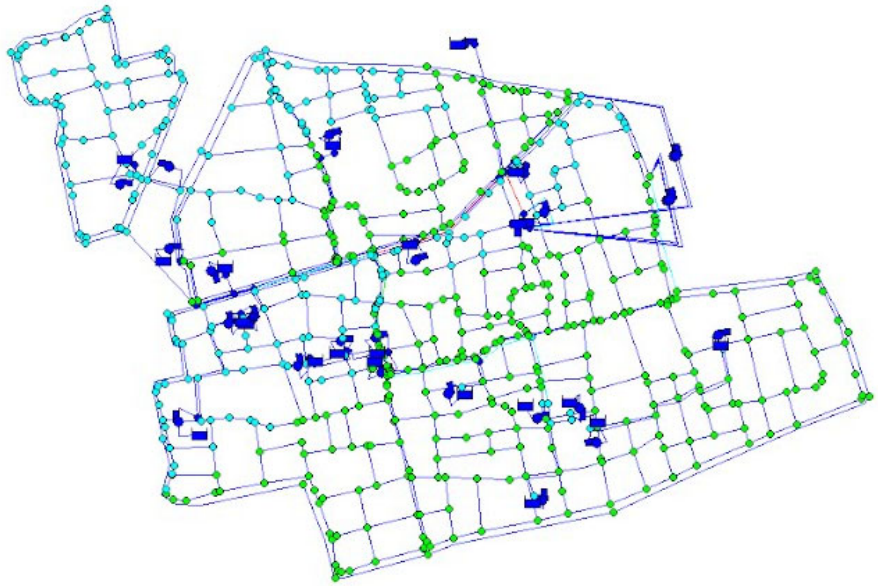
- More you already have, less you gain from a new element
- News apps example: $f(\emptyset) = 0,\ f(\{1\}) = 5,\ f(\{2\}) = 3,\ f(\{1,2\}) = 6$

$$f(\{1,2\}) - f(\{1\}) = 1 \leq 3 = f(\{2\}) - f(\emptyset)$$

$i = 2$ $A = \{1\}$ $B = \emptyset$

Submodularity in Sensing a Water Distribution Network

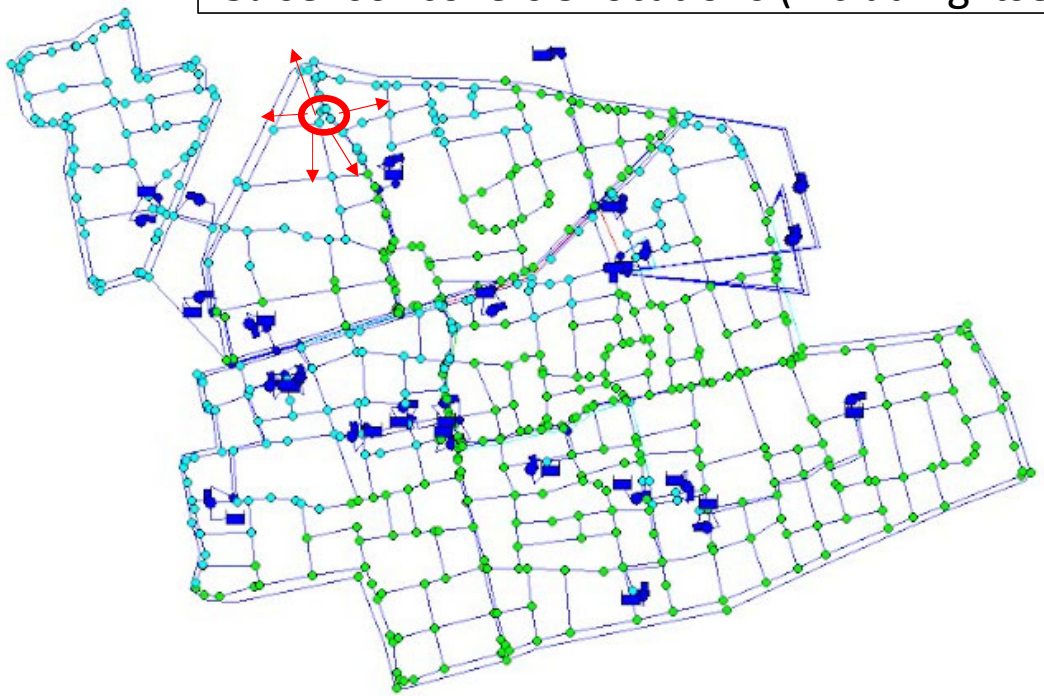
- Place sensors to detect contamination
- A sensor at node senses all neighbors
- Find optimal placement of k sensors
  - Optimal = maximum network coverage



$f(\{1\}) = 5$

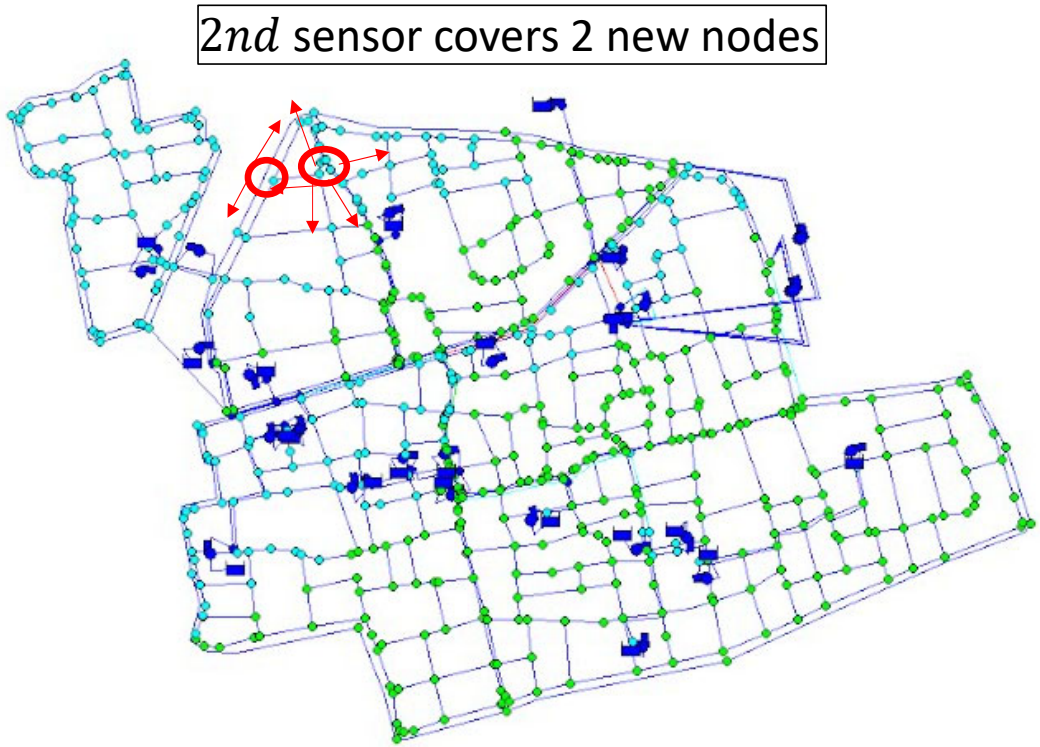
Placing First Sensor

$1st$ sensor covers 5 locations (including itself)



Value Added by Second Sensor

$f(\{1\}) = 5$
$f(\{1,2\}) = 7$

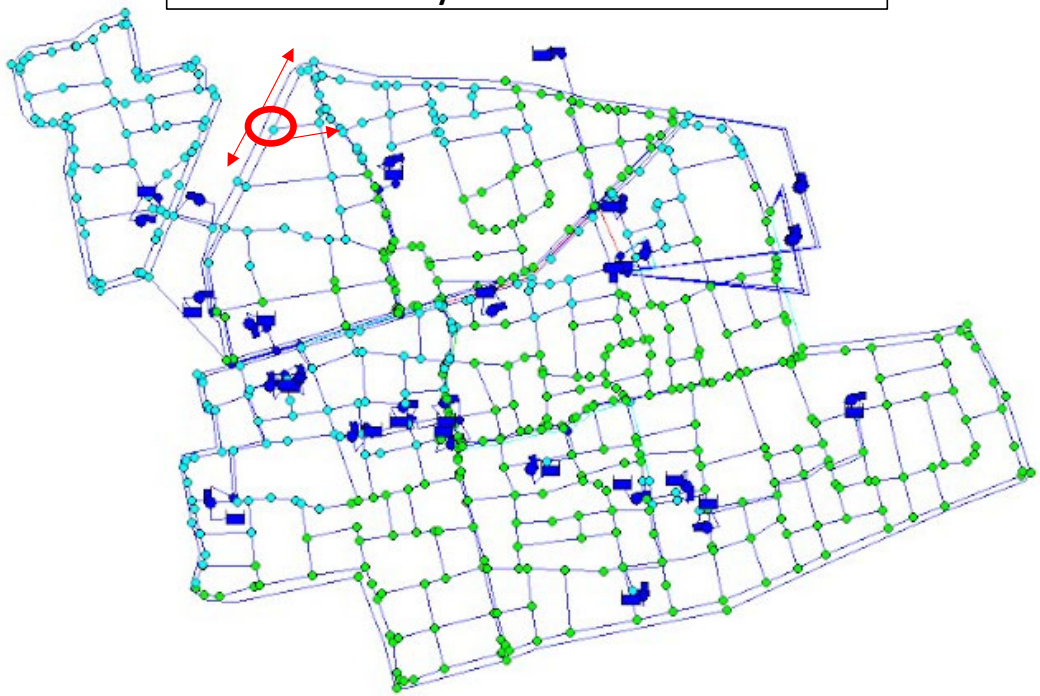


Individual Value of Second Sensor

$f(\{1\}) = 5$
$f(\{1,2\}) = 7$
$f(\{2\}) = 3$

$f(\{1,2\}) - f(\{1\}) \leq f(\{2\})$

This sensor by itself covers 3 nodes!



2. Submodular Functions

- Functions that exhibit the following property:

$$f(A \cup \{i\}) - f(A) \leq f(B \cup \{i\}) - f(B) \quad \forall B \subseteq A, i \in N \backslash \mathrm{A}$$

- Example 1 (Water network) with two locations $N = \{1,2\}$
  - $f(\emptyset) = 0,\ f(\{1\}) = 5, f(\{2\}) = 3,\ f(\{1,2\}) = 7$

$$f(\{1\} \cup \{2\}) - f(\{1\}) \leq f(\{2\}) - f(\emptyset)$$

$i = 2 \qquad A = \{1\} \qquad\qquad B = \emptyset$

2. Submodular Functions

- Functions that exhibit the following property:

$$f(A \cup \{i\}) - f(A) \leq f(B \cup \{i\}) - f(B) \quad \forall B \subseteq A, i \in N \backslash \mathrm{A}$$

- Example 2: $N = \{1,2\}$
  - $f(\emptyset) = 0, f(\{1\}) = 5,\ f(\{2\}) = 3, f(\{1,2\}) = 7$

- Example 3: $f(S) = |S|$
  - $f(S \cup \{i\}) - f(S) = f(\{i\}) \quad \forall\, i \in N, S \subseteq N$

Modular ⊂ Submodular Functions

- Functions that exhibit the following property:

$$f(A \cup \{i\}) - f(A) \leq f(B \cup \{i\}) - f(B) \quad \forall B \subseteq A, i \in N \backslash A$$

- Example 2: $N = \{1,2\}$
  - $f(\emptyset) = 0, f(\{1\}) = 5, \ f(\{2\}) = 3, f(\{1,2\}) = 7$

- Example 3: $f(S) = |S|$
  - $f(S \cup \{i\}) - f(S) = f(\{i\}) \quad \forall\, i \in N, S \subseteq N$

All modular functions are submodular

$\phi(S)$ is Submodular for RCM

Submodularity:

$$\phi(A \cup \{i\}) - \phi(A) \leq \phi(B \cup \{i\}) - \phi(B)$$

for any two assortments $B \subseteq A$ and $i \in [n] \backslash A$

$\phi(S)$ is Submodular for RCM

Submodularity:

$$\phi(A \cup \{i\}) - \phi(A) \leq \phi(B \cup \{i\}) - \phi(B)$$

for any two assortments $B \subseteq A$ and $i \in [n] \backslash A$

- Example: 2 products, $\Theta = \{1\ 2\ 0, 0\ 1\ 2, 1\ 0\ 2, 2\ 1\ 0, 0\ 2\ 1, 2\ 0\ 1\}$
  - Rank order probabilities $\{0.5, 0, 0, 0, 0, 0.5\}$
  - i.e., $\lambda_{1\ 2\ 0} = \lambda_{2\ 0\ 1} = 0.5$ and all other probabilities are 0
  - $A = \{1\}, B = \emptyset,\ i = 2$

$$\phi\ (\{1,2\}) - \phi(\{1\}) = 0.5 \leq 1 = \phi(\{2\}) - \ \phi(\emptyset)$$

Why remember Submodularity?

- Submodular structure appears in many practical problems
- Greedy algorithm **guaranteed to perform** well if objective is submodular
  - Up next
- Similar to convexity (concavity) for functions over continuous space

Monotone Submodular Maximization

With Cardinality Constraint

Classic Combinatorial Problem

$$\max_{S \subseteq N, |S| \leq k} f(S) = \max_{S \subseteq N, |S| = k} f(S)$$

- First idea: pick best k elements
- What is best?
  - Highest values of $f(e)$?

Classic Combinatorial Problem

$$\max_{S \subseteq N, |S| \leq k} f(S) = \max_{S \subseteq N, |S| = k} f(S)$$

- First idea: pick best k elements
- What is best?
  - Highest values of $f(e)$?
  - Could go very wrong,
    - Example: assortment optimization heuristic 1 last lecture: $f(\{i\}) = \phi(i, \{i\})$

E.g.: $\lambda_{1234..10,0} = 0.1,\ \lambda_{11,0} = \lambda_{12,0} = \lambda_{13,0} = \cdots = \lambda_{19,0} = 0.09$
Algorithm picks $\{1,2,3, \ldots 10\}$: $\phi(\{1,2, \ldots 10\}) = 0.1$
Optimal: $\phi(\{1,11,12,13, \ldots 19\}) = 0.91$

Greedy Algorithm

$$\max_{S \subseteq N, |S| \leq k} f(S) = \max_{S \subseteq N, |S| = k} f(S)$$

- Idea: Pick best but *accounting for past steps*
- Define marginal value function,

$$\Delta f(e|S) = f(S + \{e\}) - f(S)$$

- We can rewrite submodularity using this new notation as:

$$f(A \cup \{i\}) - f(A) = \Delta f(i|A) \leq \Delta f(i|B) = f(B \cup \{i\}) - f(B) \quad \forall B \subseteq A, i \in N \backslash \mathrm{A}$$

Greedy Algorithm

$$\max_{S \subseteq N, |S| \leq k} f(S) = \max_{S \subseteq N, |S| = k} f(S)$$

- Idea: Pick best but *accounting for past steps*
- Define marginal value function,

$$\Delta f(e|S) = f(S + \{e\}) - f(S)$$

1. **Build solution in k steps**
2. **Always add element with maximum marginal value to current set**

Add $\text{argmax}_{e \in N \setminus S} \Delta f(e|S)$

Example

Add $\text{argmax}_{e \in N \setminus S} \Delta f(e|S)$

- $N = \{1, 1', 2, 3\}$ and $k = 3$
- $f(\{1\}) = f(\{1'\}) = 10, \quad f(\{2\}) = 5, \quad f(\{3\}) = 2$
- $f(\{1,1'\}) = 10, f(\{1,2\}) = 15$
- $f(\{1,2,3\}) = 16$

Example

Add $\text{argmax}_{e \in N \setminus S} \Delta f(e|S)$

- $N = \{1, 1', 2, 3\}$ and $k = 3$
- $f(\{1\}) = f(\{1'\}) = 10, \quad f(\{2\}) = 5, \quad f(\{3\}) = 2$
- $f(\{1,1'\}) = 10, f(\{1,2\}) = 15$
- $f(\{1,2,3\}) = 16$

- Step 1 of Greedy?

Example

- $N = \{1, 1', 2, 3\}$ and $k = 3$
- $f(\{1\}) = f(\{1'\}) = 10, \quad f(\{2\}) = 5, \quad f(\{3\}) = 2$
- $f(\{1,1'\}) = 10, f(\{1,2\}) = 15$
- $f(\{1,2,3\}) = 16$

- Step 1 of Greedy: Add element 1
- Step 2?

Example

- $N = \{1, 1', 2, 3\}$ and $k = 3$
- $f(\{1\}) = f(\{1'\}) = 10, \quad f(\{2\}) = 5, \quad f(\{3\}) = 2$
- $f(\{1,1'\}) = 10, f(\{1,2\}) = 15$
- $f(\{1,2,3\}) = 16$

- Step 1 of Greedy: Add element 1
- Step 2: Add element 2

$$\Delta f(2\,|\{1\}) = 5, \qquad \Delta f(1'|\{1\}) = 0, \qquad \Delta f(3|\{1\}) \leq 2$$

Example

- $N = \{1, 1', 2, 3\}$ and $k = 3$
- $f(\{1\}) = f(\{1'\}) = 10, \quad f(\{2\}) = 5, \quad f(\{3\}) = 2$
- $f(\{1,1'\}) = 10, f(\{1,2\}) = 15$
- $f(\{1,2,3\}) = 16$

- Step 1 of Greedy: Add element 1
- Step 2: Add element 2
- Step 3: Add element 3

$$\Delta f(3|\{1,2\}) = 1, \qquad \Delta f(1'|\{1,2\}) = 0 \quad (\text{why?})$$

Example

- $N = \{1, 1', 2, 3\}$ and $k = 3$
- $f(\{1\}) = f(\{1'\}) = 10, \quad f(\{2\}) = 5, \quad f(\{3\}) = 2$
- $f(\{1,1'\}) = 10, f(\{1,2\}) = 15$
- $f(\{1,2,3\}) = 16$

- Step 1 of Greedy: Add element 1
- Step 2: Add element 2
- Step 3: Add element 3

$$\Delta f(3|\{1,2\}) = 1, \qquad \Delta f(1'|\{1,2\}) = 0 \quad (\text{why?})$$

How “Good” is Greedy?

- We know it is not optimal
  - Example from probability maximization earlier today
- How “bad” can it be?
  - Need a metric to measure worst case performance

Approximation Guarantee
[A Metric for "How Good?"]

- Let ALG be an **efficient** heuristic for the problem
  - Such as Greedy

- Let OPT be the (intractable to compute) optimal

- **Approximation guarantee** of ALG is:

$$\min_{I} \frac{\mathrm{ALG}(I)}{\mathrm{OPT}(I)} \qquad (I \text{ is problem instance})$$

Difference from competitive ratio: *gap due to computation, not due to uncertainty*

Greedy: Best Possible Guarantee

- Greedy is $\left(1 - \frac{1}{e}\right) \approx 0.63$ approximate when f is monotone and submodular
  - i.e., Greedy always outputs a solution with value at least 63% of OPT
  - True for probability maximization too

- **No efficient algorithm can beat this**

- **Performs exceedingly well in practice**
  - >90% of "optimal" in many applications